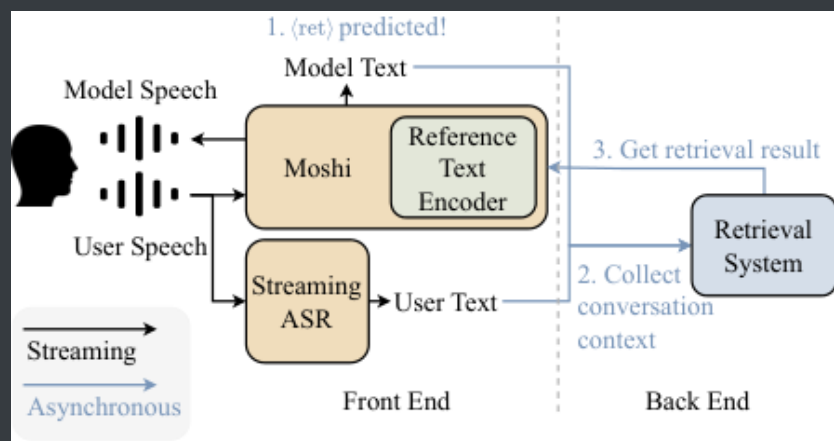


SpecGen

1. Four core sources: what work moves earlier?

1.1 MoshiRAG: feeding background knowledge into an ongoing speech model

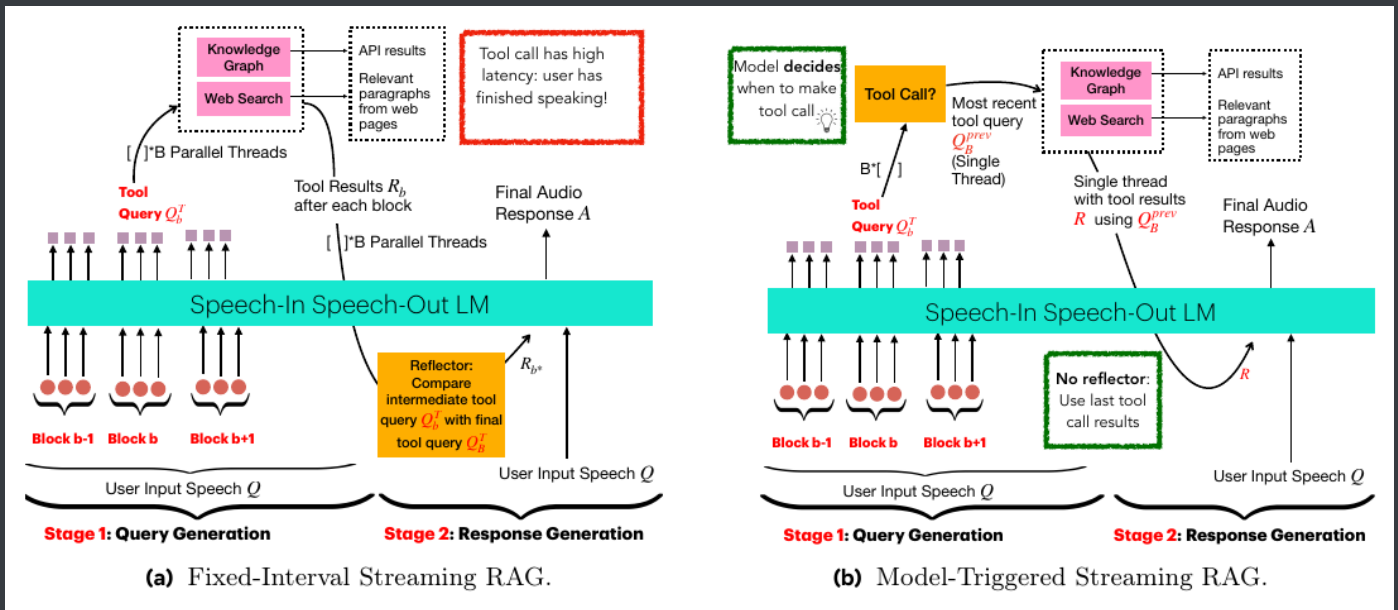


1. **What triggers retrieval?** The trained Moshi model emits `<ret>` in its text stream.
2. **What does the backend receive?** The system collects user ASR and model text, waiting 0.5 seconds for ASR to catch up. The backend can generate a concise reference with an LLM or search through Tavily.
3. **How does information return?** A Reference Text Encoder compresses and projects the reference into the speech model's representations. The frontend keeps running while retrieval is pending.

The implementation uses 7B Moshi and a separate 1B streaming ASR model; the reference encoder compresses by a factor of four. The background LLM is not directly equivalent to our natural-language query Builder. The paper does not describe a separate final-STT versus pre-query equivalence checker. [§3.3](#), particularly [§3.3.3](#)

Interpretation: The useful idea here is how asynchronous knowledge updates enter an ongoing conversation. This architecture does not supply a cheap checker that can simply be placed in front of a black-box text endpoint. Adopting its scheduling ideas does not require adopting the entire audio model; adopting its reference encoder is a separate modeling project.

1.2 Stream RAG: generate tool queries first, then answer using tool results



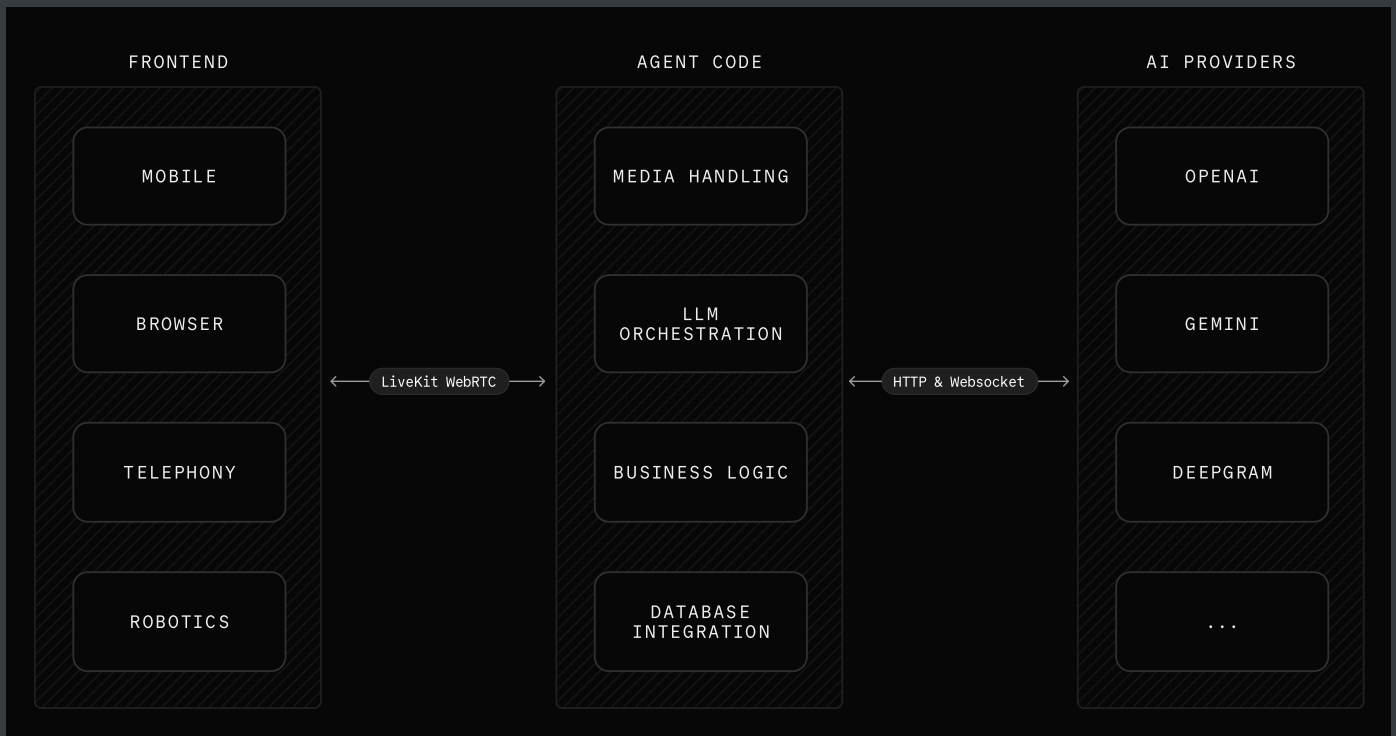
Both variants have two stages: accumulated speech $\rightarrow$ tool query; complete speech and tool text $\rightarrow$ spoken answer. They differ in how query tasks are managed:

Variant	While the user is speaking	After the user finishes
Fixed-interval	Generate queries at fixed block intervals; keep parallel tool tasks	The reflector compares against the final query and retains the earliest task with matching results
Model-triggered	Generate a new query or NO_QUERY from accumulated input and the last valid query	Use the latest tool result; no external reflector

The first uses **matching top-5 web documents / identical KG results**, rather than cosine similarity or a general LLM judge. In the second, a new query replaces the previous task; NO_QUERY can mean that the previous query remains valid. Training uses pseudo-labels from partial ASR and 10% corrupted previous queries to teach recovery. §3.1–3.2

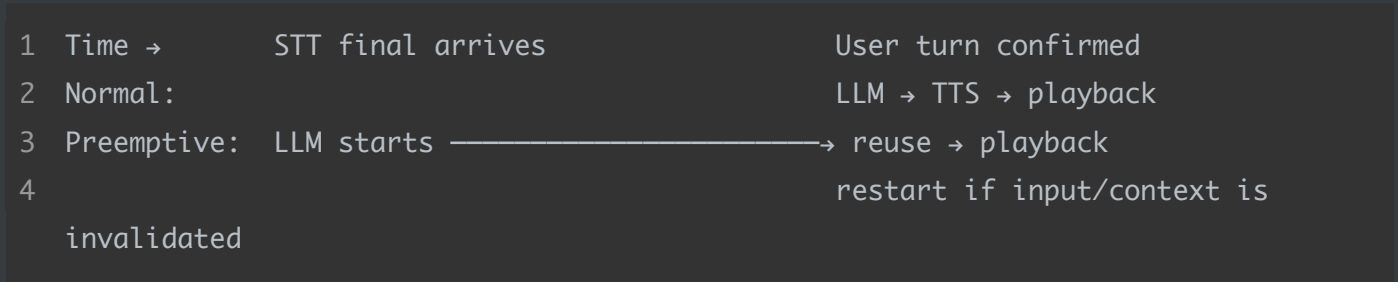
Interpretation: The fixed-interval reference is what the final query retrieves, not a human-verified answer. If the final query omits a condition, matching results cannot restore it. Comparing final retrieval results also costs time; savings come from work already overlapped, such as subsequent webpage processing. Acceptance is not free. The model-triggered variant moves retain/update decisions into a trained generation policy; adding wait to an ordinary Builder prompt does not reproduce that policy.

1.3 LiveKit: start response computation before turn confirmation



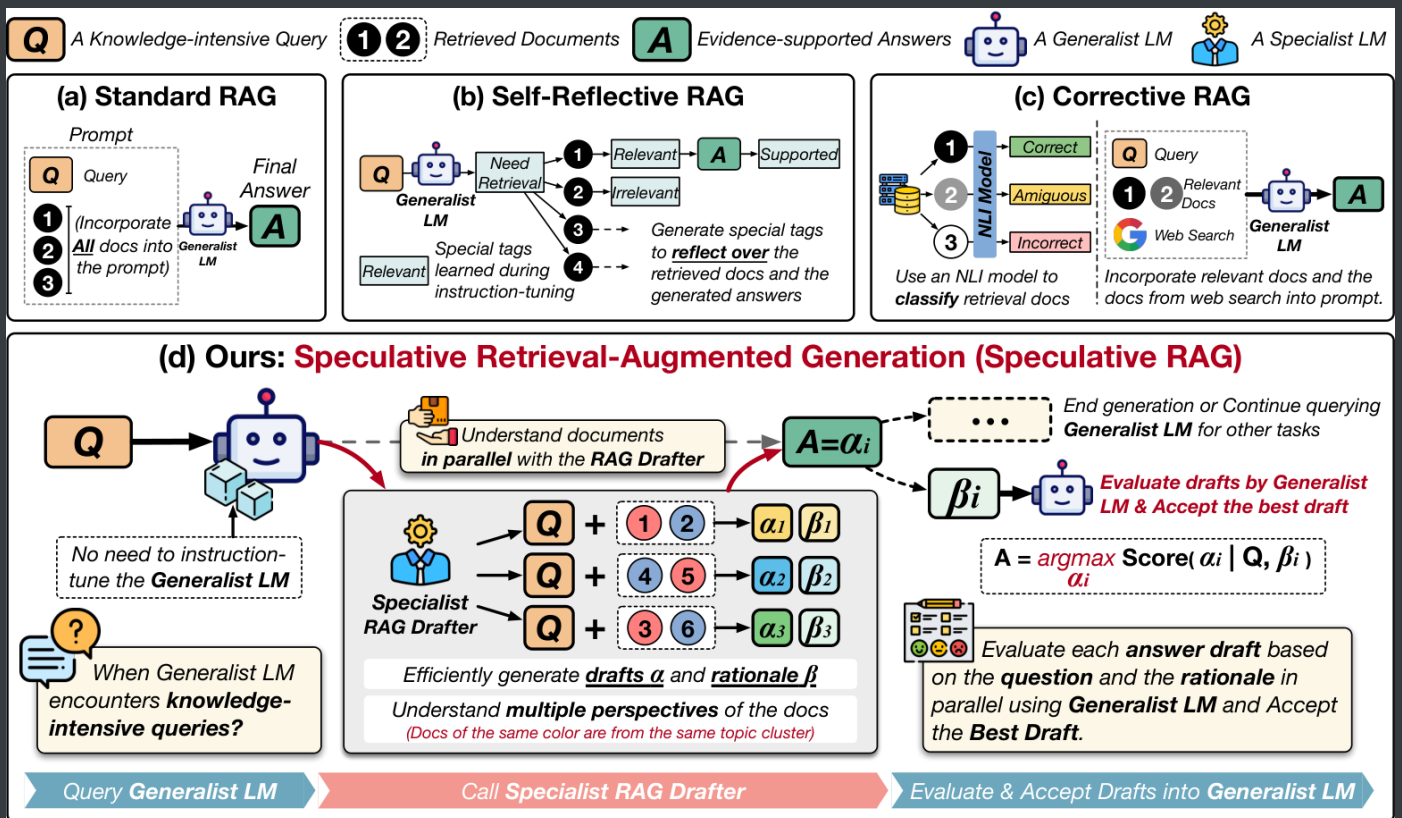
Preemptive generation starts LLM work after an STT final transcript arrives but before the user turn is confirmed. Current documentation enables it by default; speculative TTS is a separate option. If `on_user_turn_completed` changes context or tools, existing generation may be discarded and restarted. [Turn tuning](#), [preemptive speech generation](#)

The following is **our explanatory diagram, not an official figure**:



Interpretation: A final STT segment and confirmation that the user has finished the entire turn are different events. Even short questions may leave a turn-confirmation window, so this approach does not first need to predict missing words. Agent Code is our integration point in the diagram. The actual SDK version, STT provider, and hook behavior still need validation in the company project. Framework invalidation does not establish factual answer correctness.

1.4 Speculative RAG: draft answers in parallel after retrieval



Read panel **(d)** of [Figure 1, v2](#):

1. A complete question and retrieved documents are available. Documents are clustered; subsets sample across clusters.
2. A smaller, instruction-tuned specialist generates an answer draft α and rationale β for each subset in parallel.
3. A generalist verifier scores the candidates and selects the best draft. See C4 for the scoring mechanism. §3.1–3.3, [Algorithm 1](#)

Interpretation: This optimizes work after retrieval; it does not predict the remainder of a spoken question. In our terminology, its drafter is an answer Generator, and its verifier checks answer candidates. There is no direct counterpart to our partial-input query Builder. It is relevant to a slow RAG backend if we can modify that backend; it cannot directly optimize an external service whose only exposed operation is “send a question, receive an answer stream.”

2. Compare architectural responsibilities across sources

Distinguish three objects: **the retrieval question q** , **the executable query/SQL z** , and **the answer a** . Generating q does not mean z exists; executable z does not mean a answers the user. A “Checker” below may be a rule, a training label, a scorer, or a task state machine rather than a model.

B1. Builder: predict what the user will say

Work	Input → output	Implementation and supervision	Implication for this project (our assessment)
Personalized Predictive ASR, §2-3	ASR prefix and personalized history → complete utterance candidates	149M two-layer word-level LSTM; beam size 4; also historical prefix matching	The speculative object can be complete text, without generating SQL directly
Mori 2025: Dialogue Response Prefetching Based on Semantic Similarity and Prediction Confidence of Language Model, §2-4	Belief state, up to four historical utterances, partial input, response example → predicted utterance	Qwen2.5-14B-Instruct with LoRA; experiments use gold transcripts	Low training cost does not mean a small inference model; results do not cover real STT revision noise
Ohagi 2024, §3	Dialogue history → N pairs of future user utterances and responses	GPT-3.5-turbo generates both together	Next-turn pre-generation is not validation of a current-turn partial Builder
Chirpy Cardinal / Kingfisher, §4.3	History → short future reply candidates	BlenderBot 3B with PEFT; 20 samples, most frequent top-3, plus a yes candidate	Suits guided short replies; does not establish predictability of independent business questions

B2. Builder: construct tool queries or executable plans

Construction method	Source and implementation	How to interpret the output
Generate queries directly from audio	Stream RAG, §1.2	Tool input q, not a final answer
Generate programs from prefixes	<u>Online Semantic Parsing</u> , §2–4: utterance prefix → program graph → subgraphs eligible for early execution	A task graph; arbitrary SQL fragments cannot be assumed independently executable
Generate SQL from prefixes	<u>Prefix-to-SQL</u> , §3–6: prefix and schema → SQL candidates	A direct precedent, but SAVE@K measures input-token lead, not measured voice latency
Update tool plans as input arrives	<u>Speculative Interaction Agents</u> , §3.2: an LLM emits tool calls with IDs, dependencies, and \$REMOVE	Revisable plans; the runtime handles cancellation and dependency propagation
Start tools early with complete input	<u>Client-side speculative tool calls</u> , §3.1: a small model runs alongside the main model and starts candidate tools	Input is already complete; this differs from predicting unspoken constraints

B3. Builder: use existing knowledge to improve generation

Type	Implementation	Common source of confusion
Retrieve SQL examples	ReFSQL, §3.1–3.2 : SQL structure supervises the retriever; schema-link graphs and RGCN support reranking; a T5 generator uses retrieved examples and contrastive learning	Useful examples are not necessarily reusable QA answers; this is not an online partial-input checker
Select SQL examples	DAIL-SQL, Appendix A.1 : all-mpnet-base-v2 embeds masked questions; Jaccard similarity compares preliminary SQL skeletons	Preliminary SQL is already required; total cost exceeds one vector lookup
Ground requests in the database	CHESS, §3.1 : retrieve keywords, actual values and their columns, and catalog descriptions; optionally select schema; then generate SQL	Addresses plausible wording that does not map to the database, rather than prediction readiness
Rewrite queries online	Rewrite–Retrieve–Read, §3 : T5-large rewriter, retrieval, black-box reader; reader reward trains the rewriter	Training reward is not a free deployment-time checker
Generate retrieval training data offline	Promptagator, §3 : a few examples guide FLAN to generate document-query pairs for retriever/reranker training	Few-shot refers to guidance for data generation; it does not establish sufficiency of our 200 records
Expand the index offline	Doc2query, §2 : generate possible queries from documents and append them to the retrieval index	No generator call is required for every partial input

Our assessment: B1–B3 can be combined, but their contributions need separate measurement. Reusing a retrieved QA answer and using its SQL as a generation example are separate branches. Generation still adds value in the second branch because the new question may change the entity, time range, or aggregation.

C1. Checker: is speculative execution worth starting?

These decisions occur before the final input is known and usually output launch/wait. They predict future outcomes; they cannot directly inspect constraints that have not yet arrived.

Source	What is compared or predicted?	Implementation	What it cannot replace
Personalized Predictive ASR §2–3	Whether a predicted utterance will match final ASR	Small dense confidence ensemble using probability, beam rank, length, time, and historical frequency; at most one prefetch per turn	Factual answer checking; final reuse also involves matching the actual utterance
Mori 2025 §2–3	Whether predicted and final utterances will be sufficiently similar	Multilingual BERT CLS binary classifier; training labels come from a cosine threshold using stsb-xlm-r-multilingual	Future complete input is unavailable online. The label-similarity threshold and PCM probability cutoff are separate parameters
Endpoint Anticipation §3, §4.5.2	Whether the user will finish within a predicted window	Dual-stream audio representations and binary heads; prediction starts LLM/TTS, and actual endpoint confirmation releases cached output	A correct endpoint prediction does not establish correct content; audio caching is excluded from our PoC, so only timing analysis transfers
Context-Aware Preface §2.2–2.3	Whether enough information exists to prepare a short preface	Japanese ModernBERT 30M, previous system utterance plus current prefix; separate speaking-time control	Readiness concerns a preface, not the complete business answer

Difference from our length gate: Length checks input quantity; PCM estimates prediction reliability; EPA predicts a time window. One accuracy number cannot summarize all three. Adding a slow endpoint solely to implement a small gate is also difficult to justify.

C2. Checker: can we reuse a candidate once final input is known?

Acceptance object	Representative implementation	On acceptance / rejection	Remaining risk
Early versus final retrieval results	Stream RAG fixed-interval, §1.2	Continue the earliest matching task; terminate later tasks	The final query can also be wrong; coincidentally equal results do not establish general semantic equivalence
Actual versus predicted utterance	<u>Ohagi, §3</u> : sonoisa/sentence-luke-japanese-base-lite embeddings, cosine, argmax and threshold	Reuse the paired pre-generated answer above threshold; otherwise generate normally	Embeddings represent predicted user utterances , not bot answers compared directly against the user; similar topics can require different answers
Restricted short-utterance matching	<u>Kingfisher, §4.3</u> : exact strings / controlled yes-no regex	Reuse on a match; otherwise generate on demand	Coverage depends on dialogue predictability; context must still be retained
Tool name and arguments	<u>Client-side speculative tool calls, §3.1</u>	Reuse the future if the main model's eventual call matches; otherwise execute its requested call	Matching the main model does not establish that its tool decision is correct
Probability that a cached answer is reusable	<u>vCache, §4, Algorithms 1–2, §6</u> : per-entry similarity/correctness observations, sigmoid fitting and confidence bounds, randomized exploration/exploitation under an error budget	Exploit returns cache; explore calls the LLM and obtains answer-equivalence labels for updates	Depends on label quality, i.i.d. data, and sigmoid assumptions; neither an unconditional guarantee nor merely grid search

Correction to our earlier wording: “Similarity does not prove applicability” does not mean “the paper does not use similarity for acceptance.” Ohagi does. We should evaluate false acceptance rather than misdescribe the architecture.

C3. Checker: does the task still belong to the current turn or revision?

Source	Decision owner	Checks and actions	Relationship to semantic acceptance
LiveKit, §1.3	Framework lifecycle logic	Maintain speculative generation; invalidate and restart when relevant input or context/tools change	Does not require an additional semantic judge
Stream RAG model-triggered, §1.2	Trained query model and scheduler	Emit update/no-update; new tool tasks replace old tasks	The original has no additional reflector; one should not be added to its architecture diagram
<u>Speculative Interaction Agents</u> , §3.2	LLM updates IDs/REMOVE; runtime enforces constraints	Cancel dependent tasks, discard unconsumed invalid observations, and enforce separate commit rules for irreversible actions	Cancellation cannot retract information already read by the model, or facts already heard by the user

C4. Checker: is the SQL or answer good enough?

These components are often called verifiers, unit testers, or validators. Their inputs are already SQL or answers, rather than our pre-query.

Source	Inputs and implementation	Output / failure handling	Limits
Speculative RAG, §3.2–3.3	Generalist verifier scores draft/rationale token likelihood and support; it does not reread all retrieved documents	Combine drafter, self-consistency, and self-reflection scores; select the highest-scoring answer	Ranking has no general reject-all-and-retrieve-again threshold
CHESS, §3.1, Appendix C	CG revises using execution errors/empty results; UT uses an LLM to generate distinguishing natural-language tests and judge candidate compliance	Bounded revisions; select SQL by test scores	Unit Tester does not mechanically execute every assertion against real data; an empty result may be correct
PICARD, §2	Incrementally parse generated SQL tokens and constrain invalid continuations	Reject invalid tokens during decoding	Syntax and partial schema validity do not establish request fidelity; black-box endpoints may lack the required decoder control

Deployment difference: Speculative RAG’s forward-pass scoring is not equivalent to asking a Vertex chat endpoint to judge correctness. Access to the required token likelihoods and batching must be checked separately. CHESS can add multiple endpoint calls. Neither automatically meets our goal of cheap final acceptance.

C5. Checker: quality control outside the current online request

Location	Source and object checked	Why this is not current-turn acceptance
Training-data filtering	<u>Promptagator</u> , §3: round-trip filtering checks whether a generated query retrieves its source document	Filters query-document pairs, not user answers
Rewriter training	<u>Rewrite–Retrieve–Read</u> , §3: downstream reader reward	Shapes the policy during training; deployment does not automatically provide ground-truth labels
Background cache promotion	<u>Krites</u> , §3.2–3.3, §5: a judge receives the new prompt, cached prompt, and cached answer; approval promotes an entry into dynamic cache	Benefits future requests; evaluation uses an equivalence-class oracle and does not establish real LLM-judge precision. Still excluded from this PoC by user decision
Prefix-execution evaluation	<u>Online Semantic Parsing</u> , §4: generation confidence gates early execution; the final gold graph supports simulation	The gold program is not an available online checker; the paper’s parsing-time assumptions also need reassessment

G. Generator: what is actually generated early?

Generation stage / artifact	Source	Prerequisites	What this project should measure
Ongoing speech with later references	MoshiRAG, §1.1	Trained speech frontend and reference injection	Time to the first evidence-supported answer, not just filler
Answer after retrieval	Stream RAG, §1.2	Retained tool results and complete speech	Final answer quality and post-speech waiting time
Response generation ahead of turn confirmation	LiveKit, §1.3	STT final precedes turn confirmation; context remains reusable	Cancellation rate, net savings, first useful answer
Complete future	Ohagi / Kingfisher, B1 and C2	Future utterance matches an acceptable	Incorrect reuse and coverage,

replies		candidate	not latency only on hits
Multiple answer drafts, then scoring	Speculative RAG, §1.4 and C4	Complete question and retrieved documents	Total drafting/scoring cost; retrieval waiting remains
Background oracle text stream	<u>KAME, §2–3</u> : send partial STT periodically to a text LLM; prioritize newer responses; a trained S2S frontend consumes an additional oracle token stream	Frontend must learn to use asynchronous updates; training includes simulated oracle progression	Not equivalent to passing a new string into ordinary TTS
Separate short preface and factual answer	<u>Context-Aware Preface, §2.3</u> : gpt-4o-mini preface, gpt-4o main answer	Both preface readiness and speaking-time conditions hold	Preface latency and substantive-answer latency separately
SQL rather than the user-facing answer	ReFSQL / DAIL-SQL / CHESS, B3 and C4	Schema, examples, and grounding information	SQL semantics and execution accuracy; answer generation is still needed

3. One failure example: what can each check detect?

This is **our analytical example, not a new paper experiment**. The user first says “customers older than 60,” then corrects it to “actually, younger than 60.” Their embeddings may be close, although the filter direction is reversed.

Check	What it may observe	How to interpret it
Text-length gate	Both inputs are long enough	Permission to try, not permission to reuse
Prediction confidence	High confidence before the correction	Earlier confidence does not authorize reuse after information changes
Cosine threshold	Same topic, entity, and number	A high threshold does not mechanically protect comparison direction, negation, or units
Exact tool-argument match	<code>age > 60</code> differs from <code>age < 60</code>	Can reject if both sides are constructed correctly; can still falsely accept if both omit the constraint
Retrieval-result equality	Both queries may return the same documents	Equal document sets do not imply interchangeable answers
SQL execution / syntax check	Both SQL queries execute successfully	Does not identify which query the user intended
Semantic comparison of the complete request and candidate	Can inspect the conflicting direction	Judge misses and latency still require measurement; do not claim all constraints are guaranteed
Task revision	New input has arrived	Can prevent stale results from committing, but does not assess the new candidate's quality

Similarly, “How many Canadian customers are over 60? How many of those made a purchase last year?” contains at least two subquestions. A candidate may cover the first but not the second.

Acceptance labels should allow partial coverage; otherwise, the system may label a partially correct answer as complete, or discard useful work entirely.

Artifacts	Need What to Accept	onFailure/Existed?
Question q		final input/Existed
SQL z		SQL 生成 / 修订；不把能执行当语义正确
Reference r		Answerer 继续检索、重写或说明缺失
CacheAnswer a		_/None cache env
Task t		